

Rust → MLOG

用过程宏打造 Mindustry 汇编编译器

Typst 编译 — 2026-07-15

这不是一门新语言。

这是一个嵌入在 Rust 中的 MLOG 编译器——

从第一行 DSL 开始，到最后一条 jump 指令结束。

目录

1	序章：为什么需要 Rust → MLOG?	4
1.1	三种解决方案	4
1.2	本教程的结构	5
2	MLOG 汇编速览	7
2.1	什么是 MLOG	7
2.2	处理器类型	7
2.3	核心指令族	7
2.4	输出指令	8
2.5	单位指令	9
2.6	特殊变量	9
2.7	硬限制	9
2.8	一个完整的 MLOG 程序	9
2.9	小结	10
3	设计 DSL — 从 MLOG 到 Rust 语法	11
3.1	设计原则	11
3.2	语法映射表	11
3.3	一个完整对照示例	11
3.4	DSL 的明确限制	12
3.5	为什么不像 Mindcode 那样做一个独立语言	12
3.6	小结	13
4	过程宏架构设计	14
4.1	三种宏类型的取舍	14
4.2	Crate 结构	14
4.3	编译器数据流	15
4.4	AST 节点设计	15
4.5	宏入口	17
4.6	为什么返回 String 而不是用 quote!	17
4.7	小结	17
5	解析器 — 把 TokenStream 变成 AST	18
5.1	为什么不用 syn 的 DeriveInput	18
5.2	准备工作：Peek 和 Parse 模式	18
5.3	解析语句	18
5.4	表达式解析	19
5.5	经典的递归下降解析	20
5.6	特殊语句解析	23
5.7	解析器的位置	24
5.8	小结	24
6	中间表示 — 三地址码	25
6.1	什么是三地址码	25
6.2	IR 指令定义	25
6.3	将 AST 表达式降级为三地址码	26
6.4	示例：表达式分解	28
6.5	将语句降级为 IR	28
6.6	标签管理	31
6.7	小结	31
7	代码生成 — IR → MLOG 文本	32
7.1	IrInstr → MLOG 字符串	32
7.2	值的格式化	33
7.3	完整的代码生成示例	34

7.4	MLOG 变量名的考虑	34
7.5	一个值得注意的设计选择	34
7.6	在宏入口中串联	35
7.7	小结	35
8	控制流 – jump 的三种形态	36
8.1	if-else 的 jump 模式	36
8.2	while 循环的 jump 模式	37
8.3	loop 循环 + break	38
8.4	短路求值	38
8.5	条件跳转的条件码速查	39
8.6	小结	39
9	变量系统	40
9.1	MLOG 的变量模型	40
9.2	问题场景	40
9.3	解决方案一：扁平化	40
9.4	解决方案二：唯一名称	40
9.5	作用域管理	41
9.6	临时变量	42
9.7	不可变性检查	42
9.8	完整例子	43
9.9	小结	43
10	输出生成 – 最后的 MLOG 文本	44
10.1	生成最终的字符串	44
10.2	可选：添加注释	44
10.3	回到 macro 入口	45
10.4	用户如何使用生成的 MLOG	45
10.5	格式化细节	46
10.6	小结	46
11	错误处理与诊断	47
11.1	Span 从哪来	47
11.2	语义错误 vs 语法错误	47
11.3	诊断系统	47
11.4	典型语义检查	48
11.5	与 syn 的错误处理集成	49
11.6	调试模式	50
11.7	小结	50
12	实战：从 DSL 到 MLOG 的完整流程	51
12.1	完整的宏入口	51
12.2	场景一：仓库库存监控	51
12.3	场景二：单位巡逻	52
12.4	测试编译器的输出	54
12.5	编译器的文件清单	54
12.6	小结	55
13	总结与展望	56
13.1	回顾旅程	56
13.2	概念速查表	56
13.3	与其他方案对比	56
13.4	进阶扩展	56
13.5	推荐阅读	56
13.6	最后的思考	57

1 序章：为什么需要 Rust → MLOG?

你在 Mindustry 中想实现一个智能仓库管理系统。

需要做的事：

- 扫描所有容器中的物品数量
- 当某种物品低于阈值，触发生产线
- 当存储空间不够，调度运输单位
- 在显示器上实时展示库存状态

MLOG 是一门类汇编语言——没有函数，没有类型，没有变量作用域，没有循环语法。你能用的工具有：

- `set x 5` — 给变量赋值
- `op add result a b` — 算术/逻辑运算
- `jump label condition a b` — 条件/无条件跳转
- `read / write / sensor / control / print / draw`

用这些写一个仓库管理系统，代码会长得像这样：

```
set copper 0
set lead 0
set threshold 100
:check_copper
sensor copper container1 @copper
op lt need_power copper threshold
jump skip_power equal need_power false
control enabled conveyor1 true
:skip_power
op add index index 1
op lt continue index 4
jump check_copper equal continue true
print "Copper: "
print copper
printf flush message1
end
```

当逻辑复杂时——嵌套条件、多状态机、多单位协调——手写 MLOG 很快就变成维护噩梦。

1.1 三种解决方案

1.1.1 方案一：手写 MLOG

直接编辑文本，粘贴到 Mindustry 处理器。

优点	缺点	:- :-	零依赖	无可读性	完全控制	无 IDE 支持	容易出错

1.1.2 方案二：Mindcode

Mindcode 是一个独立的编译器，把高级语言编译成 MLOG。

```
while copper < threshold do
  enabled = true;
end;
```

但它有局限性：

- 独立语言和工具链——和 Rust 项目不集成
- 不能在 Rust 代码中直接生成 MLOG
- 编译结果需要手动复制粘贴

1.1.3 方案三: Rust 过程宏嵌入式 DSL

在 Rust 源码中直接写 MLOG DSL:

```
let program = mlog! {  
    let mut copper = 0;  
    let threshold = 100;  
  
    loop {  
        copper = sensor(container1, @copper);  
        if copper < threshold {  
            enable(conveyor1);  
        }  
        if index >= 4 { break; }  
        index += 1;  
    }  
  
    print("Copper: ");  
    print(copper);  
    print_flush(message1);  
};
```

编译时, 过程宏把它转换成正确的 MLOG 文本字符串:

```
set threshold 100  
set index 0  
:__loop_0  
sensor copper container1 @copper  
op lt __tmp_0 copper threshold  
jump __skip_0 equal __tmp_0 false  
control enabled conveyor1 true 0 0  
:__skip_0  
op gteq __tmp_1 index 4  
jump __break_0 not __tmp_1 false  
op add index index 1  
jump __loop_0  
:__break_0  
print "Copper: "  
print copper  
printf flush message1
```

核心直觉

这就是本教程要做的事: 在 Rust 中写一个嵌入式编译器。

用过程宏把类 Rust 的 DSL 编译成 MLOG 汇编。编译器本身在 Rust 编译时运行, 产生的 MLOG 文本可以直接粘贴到 Mindustry。

1.2 本教程的结构

这是一次从零开始搭建编译器的旅程。

第一章: 深入理解目标语言—MLOG 的完整指令集和限制 **第二章:** 设计 DSL—MLOG 语义如何映射到 Rust 语法 **第三章:** 过程宏架构—选择宏类型, 搭项目骨架 **第四章:** 解析器—手写一个 TokenStream → AST 的解析器 **第五章:** 中间表示—三地址码 IR, 表达式分解 **第六章:** 代码生成—AST/IR → MLOG 指令序列 **第七章:** 控制流—if/while/loop 的 jump 标签策略 **第八章:** 变量系统—作用域、临时

变量、唯一名称 **第九章**：输出——把 MLOG 指令格式化为文本 **第十章**：错误诊断——如何给用户好的错误信息 **第十一章**：组装完整编译器——从 DSL 到 MLOG 的完整流程 **第十二章**：进阶扩展与总结

准备好从 Rust 走向 MLOG 了吗？

2 MLOG 汇编速览

实验记录： 第一站：目标语言

在写编译器之前，必须彻底理解目标语言。这一章覆盖 MLOG 的完整指令集和关键限制。

2.1 什么是 MLOG

MLOG 是 Mindustry 内置处理器（processor）执行的汇编式语言。

- 每个处理器有独立的指令序列，编号从 0 开始
- 每条指令占一行，无嵌套
- 执行模型：顺序执行，jump 改变执行流
- 每个 tick 执行若干条指令（取决于处理器类型）
- 变量是动态类型的——只有 number 和 null
- 没有调用栈——没有真正的函数

2.2 处理器类型

处理器	指令上限	特点
Micro Processor	128	最小，够简单逻辑
Logic Processor	500	标准
Hyper Processor	1000	最强大
World Processor	1000	全局控制（地图规则、天气等）

注意

1000 条指令看起来不少，但每条高级语言语句通常对应 2-5 条 MLOG 指令。一个中等复杂的 while 循环轻松吃掉 10+ 条指令。

2.3 核心指令族

2.3.1 set: 赋值

```
set x 5           // x = 5
set y x           // y = x
set name "hello" // name = "hello"
```

变量在首次 set 出现时自动创建。没有类型声明，任何变量可以保存数字或字符串引用。

2.3.2 op: 运算

```
op add result a b // result = a + b
op sub result a b // result = a - b
op mul result a b // result = a * b
op div result a b // result = a / b (浮点)
op idiv result a b // result = a // b (整数除法)
op mod result a b // result = a % b
op pow result a b // result = a ^ b
op eq result a b // result = a == b (比较用，不是严格相等)
op neq result a b // result = a != b
op lt result a b // result = a < b
op gt result a b // result = a > b
op lteq result a b // result = a <= b
op gteq result a b // result = a >= b
op land result a b // result = a && b (逻辑与)
op and result a b // result = a & b (位与)
op or result a b // result = a | b (位或)
op xor result a b // result = a ^ b (位异或)
op shl result a b // result = a << b
```

```

op shr result a b    // result = a >> b
op not result a 0    // result = !a (位非)
op max result a b    // result = max(a, b)
op min result a b    // result = min(a, b)
op abs result a 0    // result = |a|
op sin result a 0    // result = sin(a) 弧度
op cos result a 0    // result = cos(a)
op sqrt result a 0   // result = sqrt(a)
op rand result a 0   // result = 0..a 范围的随机整数

```

概念

MLOG 的运算指令是 **三地址码** (three-address code):

```
op <操作符> <结果> <操作数1> <操作数2>
```

这是一种接近编译器中层的表示形式——后面我们会大量使用这种形式。

2.3.3 jump: 控制流

```

jump label always      // 无条件跳转
jump label condition a b  // 条件跳转

```

条件可以是: == != < <= > >= not (布尔假时跳转) always。

```

jump end_loop gteq counter 10 // if counter >= 10 goto end_loop
jump skip equal x null        // if x == null goto skip
jump loop always              // goto loop

```

2.3.4 read / write: 内存操作

```

read result memoryCell index // result = memoryCell[index]
write value memoryCell index // memoryCell[index] = value

```

必须链接内存单元 (Memory Cell) 才能使用。index 从 0 开始。

2.3.5 sensor: 读取方块/单位属性

```
sensor result object property // result = object.property
```

sensor result @unit @x — 读取当前绑定单位的 x 坐标。 sensor result turret1 @ammo — 读取炮塔的弹药量。

2.3.6 control: 控制方块

```

control enabled block value 0 0 0 // 启用/禁用方块
control shoot block x y shoot 0   // 控制炮塔射击

```

2.4 输出指令

```

print "hello" // 添加到打印缓冲区
print counter // 添加变量的值

```



```
printflush message1      // 刷新缓冲区到消息方块

draw clear r g b         // 清屏
draw color r g b a 0 0   // 设置颜色
draw rect x y w h 0 0    // 绘制矩形
drawflush display1       // 刷新到显示器
```

2.5 单位指令

```
ubind @poly              // 绑定下一个 Poly 单位到 @unit
ucontrol move x y 0 0 0 // 移动当前单位
uradar ...               // 单位雷达
```

2.6 特殊变量

| 变量 | 含义 | |:-|:-| | @counter | 当前指令编号 | | @time | 游戏时间(秒) | | @tick | 当前 tick | | @unit | 当前绑定的单位 | | @this | 当前处理器 | | null | 空值 |

2.7 硬限制

⚠ 注意

作为编译器作者，这些限制直接影响你的设计：

1. **无调用栈**：不能 push/pop 返回地址，必须用 jump 模拟子程序
2. **无类型系统**：编译时无法做类型检查，只能靠命名惯例
3. **指令上限**：Micro 128, Logic 500, Hyper 1000
4. **变量是全局的**：DSL 中的“局部变量”需要编译器做名称转换
5. **条件跳转只能是二元比较**：不能 if x 只能 jump label eq x true
6. **单线程**：所有逻辑在一个指令流中

2.8 一个完整的 MLOG 程序

我们以最简单的例子结束：

```
// 打印 1 到 5
set counter 1
set limit 5
:loop
print counter
op add counter counter 1
op lteq continue counter limit
jump loop not continue false
printflush message1
```

输出：

```
1
2
3
4
5
```

MLOG 的语法很简单——但用这些原语组装复杂逻辑，就是编译器的工作了。

2.9 小结

- MLOG 是类汇编语言: `set / op / jump` 三大家族
- `op` 指令是三地址码: `op <opcode> <dest> <a> <b>`
- `jump` 是唯一控制流手段: 条件/无条件跳转到标签
- 无调用栈、无类型、无作用域
- 输出靠 `print/draw` 缓冲区 + `flush`
- 处理器有严格指令上限
- 这些限制 正是编译器要帮助开发者管理的

3 设计 DSL — 从 MLOG 到 Rust 语法

🔧 实验记录：第二站：语法映射

现在设计 DSL。目标：写起来像 Rust，编译出去像 MLOG。

3.1 设计原则

📖 概念

三条原则指导 DSL 设计：

1. 表面像 **Rust**：let、if、while、loop 等关键词都沿用
2. 映射可预测：每个 DSL 构造对应一组确定的 MLOG 指令
3. 编译时检查：变量未定义、break 在非循环中等错误在编译时捕获

3.2 语法映射表

3.2.1 变量

DSL	MLOG	:- :-	let x = 5; set x 5 let mut x = 0; set x 0 x = 10; set x 10 let y = x; set y x
-----	------	--------	---

变量自动创建，首次出现即声明。let / let mut 在语义上有区别（不可变 vs 可变），但编译后都是 set——不变性检查在编译时做。

3.2.2 算术和比较

DSL	MLOG	:- :-	let z = x + y; op add z x y let z = x * y + 1; op mul __tmp x y 然后 op add z __tmp 1 let b = x > y; op gt b x y let b = x == 0; op eq b x 0
-----	------	--------	--

复杂表达式自动分解——这是编译器的责任。

3.2.3 条件

DSL	MLOG	:- :-	if x > 0 { ... } op gt __tmp x 0 → jump __skip not __tmp if x > 0 { ... } else { ... } 同上 + 额外 jump 到 else 分支
-----	------	--------	---

3.2.4 循环

DSL	MLOG	:- :-	while x < 10 { ... } 标签循环头 + 条件检查 + 条件跳转出口 loop { ... break; } 无条件循环 + break 转条件跳转 for i in 0..5 { ... } 展开为 while 循环
-----	------	--------	---

3.2.5 特殊操作

DSL	MLOG	:- :-	print("hello"); print "hello" print(x); print x print_flush(msg); printf flush msg let v = sensor(@unit, @x); sensor v @unit @x enable(block); control enabled block 1 0 0 0 disable(block); control enabled block 0 0 0 0 ubind(@poly); ubind @poly
-----	------	--------	--

3.3 一个完整对照示例

DSL:

```
let mut i = 0;
let limit = 5;
let mut sum = 0;

while i < limit {
    sum += i;
    i += 1;
}
```

```

}

print("Sum: ");
print(sum);
print_flush(message1);

```

期望 MLOG 输出:

```

set i 0
set limit 5
set sum 0
:__while_0
op lt __tmp_0 i limit
jump __end_while_0 not __tmp_0
op add sum sum i
op add i i 1
jump __while_0
:__end_while_0
print "Sum: "
print sum
printf flush message1

```

🔍 核心直觉

不是 1:1 翻译，也不是完全不一样——

每条 DSL 语句对应 1-3 条 MLOG 指令。表达式的自动分解和处理是编译器的核心工作。

因为 MLOG 是「三地址码」形式的，它天然适合做编译目标。

3.4 DSL 的明确限制

⚠️ 注意

设计上不支持的特性:

1. 函数调用 — MLOG 没有调用栈，所有代码内联展开
2. 字符串操作 — MLOG 不支持字符串运算，只能 print
3. 数组和结构体 — 只能用内存单元模拟
4. 递归 — 不可能，因为没有栈
5. 闭包/高阶函数 — 超出 MLOG 的能力
6. 浮点精度控制 — MLOG 的数字是 64 位浮点

这不是 Rust 的子集——这是一个受限的嵌入式 DSL。它的表达能力受 MLOG 指令集的约束。

3.5 为什么不像 Mindcode 那样做一个独立语言

Mindcode 选择做独立编译器，因为它追求通用性。我们的 DSL 选择嵌入 Rust，是因为:

1. **Rust 工具链**: IDE 高亮、格式化、Git 比较都来自 Rust
2. **编译时执行**: 宏在 cargo build 时运行——不增加运行时依赖
3. **混合代码**: MLOG 程序可以和 Rust 逻辑混合在一起
4. **文本输出**: 展开为 &str，可以写文件、网络发送、嵌入常量

这是 proc macro 作为编译器前端的经典应用。

3.6 小结

- DSL 语法映射到 MLOG 指令必须可预测、直接
- 变量映射到 `set`，运算映射到 `op`，控制流映射到 `jump`
- 特殊操作 (`sensor`、`control`、`print`) 有专用函数
- 不支持函数、字符串运算、递归—受 MLOG 限制
- 嵌入 Rust 的好处：IDE 支持、编译时执行、零运行时成本

4 过程宏架构设计

🔪 实验记录：第三站：选型与项目骨架

开始编码之前，先确定整体架构。选择哪种过程宏、怎么组织代码、数据流怎么走——这些决定直接影响到后续所有章节。

4.1 三种宏类型的取舍

在 `proc-macro-guide` 中我们学过了三种过程宏。现在为 `MLOG` 编译器选择一个。

| 方案 | 用法 | 适合吗 | |:-|:-|:-| | `Derive` 宏 | `#[derive(Mlog)] struct Foo` | ❌
语法不匹配，我们需要代码块 | | `Attribute` 宏 | `#[mlog] fn foo() { ... }` | 😞 可行，但
函数签名是多余的 | | 函数式宏 | `mlog! { ... }` | ✅ 最自然，直接嵌入代码块 |

选择函数式宏：`mlog! { ... }` 最直观。输入是一段 DSL 代码块，输出是 `MLOG` 文本的字符串字面量。

```
let program: &str = mlog! {  
    let mut i = 0;  
    while i < 10 {  
        print(i);  
        i += 1;  
    }  
};  
// program 是展开后的 MLOG 文本
```

4.2 Crate 结构

```
mlog/  
├─ Cargo.toml           # facade crate (用户直接依赖)  
├─ src/  
│   └─ lib.rs           # 重新导出宏 + 工具函数  
└─ mlog-macro/  
    ├─ Cargo.toml       # proc-macro = true  
    └─ src/  
        ├─ lib.rs        # 宏入口  
        ├─ parser.rs     # DSL 解析器  
        ├─ ast.rs        # 语法树定义  
        ├─ ir.rs         # 三地址码中间表示  
        ├─ codegen.rs    # IR → MLOG 代码生成  
        └─ diagnostics.rs # 错误报告
```

```
// mlog/Cargo.toml  
[dependencies]  
mlog-macro = { path = "../mlog-macro" }
```

```
// mlog-macro/Cargo.toml  
[lib]  
proc-macro = true  
  
[dependencies]  
proc-macro2 = "1.0"  
quote = "1.0"
```

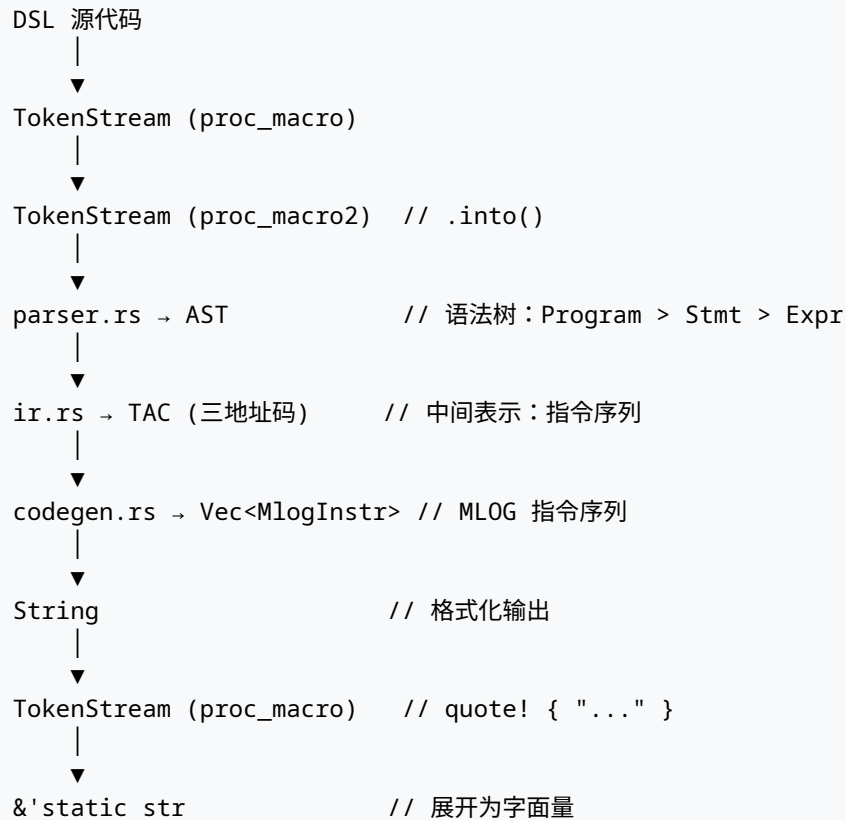


概念

Facade 模式: 用户只依赖 `mlog crate`。mlog 重新导出 `mlog!` 宏。

`mlog-macro` 是实际的 `proc-macro crate`，用户不直接依赖它。这是 Rust 过程宏生态的标准做法。

4.3 编译器数据流



4.4 AST 节点设计

```
// ast.rs
pub struct Program {
    pub stmts: Vec<Stmt>,
}

pub enum Stmt {
    Let {
        name: Ident,
        mutable: bool,
        init: Expr,
    },
    Assign {
        name: Ident,
        value: Expr,
    },
    If {
        condition: Expr,
        then_branch: Vec<Stmt>,
    },
}
```

```

        else_branch: Option<Vec<Stmt>>,
    },
    While {
        condition: Expr,
        body: Vec<Stmt>,
    },
    Loop {
        body: Vec<Stmt>,
    },
    Break,
    Print(Expr),
    PrintFlush(Ident),
    Enable(Ident),
    Disable(Ident),
    Sensor {
        result: Ident,
        object: Ident,
        property: Ident,
    },
    ExprStmt(Expr),
}

pub enum Expr {
    Number(f64),
    String(String),
    Variable(Ident),
    BinaryOp {
        lhs: Box<Expr>,
        op: BinaryOp,
        rhs: Box<Expr>,
    },
    UnaryOp {
        op: UnaryOp,
        expr: Box<Expr>,
    },
}

pub enum BinaryOp {
    Add, Sub, Mul, Div, Mod,
    Eq, Neq, Lt, Gt, Lteq, Gteq,
    And, Or,
}

pub enum UnaryOp {
    Neg, Not,
}

```

核心直觉

AST 是编译器的第一层抽象。

它不关心 MLOG 的三地址码限制——`let z = x + y * 3` 在 AST 中是一个嵌套的 BinaryOp 树。后面 IR 阶段会把嵌套表达式拍平成 MLOG 能消费的三地址码序列。

4.5 宏入口

```
// mlog-macro/src/lib.rs
use proc_macro::TokenStream;

mod ast;
mod parser;
mod ir;
mod codegen;
mod diagnostics;

#[proc_macro]
pub fn mlog(input: TokenStream) -> TokenStream {
    // 1. 解析 DSL → AST
    let program = match parser::parse_program(input.into()) {
        Ok(ast) => ast,
        Err(err) => return err.to_compile_error().into(),
    };

    // 2. AST → IR (三地址码)
    let ir = ir::lower_program(&program);

    // 3. IR → MLOG 指令序列
    let mlog_instrs = codegen::generate(&ir);

    // 4. 格式化为字符串
    let output = codegen::format(&mlog_instrs);

    // 5. 返回为字符串字面量
    quote::quote! { #output }.into()
}
```

4.6 为什么返回 **String** 而不是用 **quote!**

注意第 5 步——`quote! { #output }` 生成的是 `TokenStream` 形式的字符串字面量。

这和之前教程中的做法不同。之前我们用 `quote!` 生成 Rust 代码。这里我们生成的 MLOG 不是 Rust 代码——它只是一个字符串常量，可以嵌入 Rust 程序中使用。



概念

编译器的输出是 **MLOG 文本**，不是 Rust 代码。

`mlog! { ... }` 展开后是 `"set x 5\nop add ..."` 这样的字符串。用户可以用 `include_str!`、`write!` 等方式进一步处理。

4.7 小结

- 选函数式宏 `#proc_macro`，最自然地嵌入 DSL 代码块
- Facade crate 模式: `mlog + mlog-macro`
- 编译器流水线: `TokenStream → AST → IR → MLOG 指令 → String`
- AST 是嵌套语法树，IR 是三地址码，两个阶段分离关注点
- 输出是 MLOG 文本字符串，不是 Rust 代码
- 模块化: `parser / ast / ir / codegen / diagnostics` 各司其职

5 解析器 — 把 TokenStream 变成 AST

 实验记录：第四站：手写解析

现在写解析器。我要把 `mlog! { ... }` 中的 DSL 代码解析成第三章定义的那些 AST 节点。

5.1 为什么不用 syn 的 DeriveInput

之前写 `derive` 宏时，`syn` 提供了 `DeriveInput`、`ItemFn` 等现成的解析结构。但这里不一样——

我们的输入不是标准 Rust 语法结构。`let mut x = 0;` 是有效的 Rust，但 `print_flush(message1);` 和 `sensor(v, @unit, @x)` 不是一——它们是自定义的 DSL。

所以需要手写解析器。

5.2 准备工作：Peek 和 Parse 模式

`syn` 仍然有用——它的 `ParseStream` 提供了 `peek/parse` 能力：

```
use syn::parse::{Parse, ParseStream};
use proc_macro2::{TokenStream, Ident, Literal, TokenTree};

// 逐 token 读取
fn parse_program(input: ParseStream) -> syn::Result<Program> {
    let mut stmts = Vec::new();
    while !input.is_empty() {
        stmts.push(parse_stmt(input)?);
    }
    Ok(Program { stmts })
}
```

核心方法：

- `input.peek(Token![let])` — 看下一个 token，不消费
- `input.parse::<Ident>()` — 消费并解析一个标识符
- `input.parse::<Token![;]>()` — 消费分号
- `input.is_empty()` — 是否到达末尾

5.3 解析语句

```
fn parse_stmt(input: ParseStream) -> syn::Result<Stmt> {
    if input.peek(Token![let]) {
        parse_let_stmt(input)
    } else if input.peek(Token![if]) {
        parse_if_stmt(input)
    } else if input.peek(Token![while]) {
        parse_while_stmt(input)
    } else if input.peek(Token![loop]) {
        parse_loop_stmt(input)
    } else if input.peek(Token![break]) {
        parse_break_stmt(input)
    } else if is_identifier(input) && input.peek2(Token![=]) {
        // x = ...; 赋值语句
        parse_assign_stmt(input)
    } else {
        // 以表达式开头的语句
        parse_expr_stmt(input)
    }
}
```

5.3.1 let 语句

```
fn parse_let_stmt(input: ParseStream) -> syn::Result<Stmt> {
    input.parse::
```

5.3.2 if 语句

```
fn parse_if_stmt(input: ParseStream) -> syn::Result<Stmt> {
    input.parse::
```

5.3.3 解析代码块

```
fn parse_block(input: ParseStream) -> syn::Result<Vec<Stmt>> {
    let content;
    syn::braced!(content in input); // 匹配 { ... }
    let mut stmts = Vec::new();
    while !content.is_empty() {
        stmts.push(parse_stmt(&content)?);
    }
    Ok(stmts)
}
```

5.4 表达式解析

表达式解析是编译器中最经典的部分——需要处理优先级和结合性。

定义

操作符优先级（从高到低）：

优先级	操作符	:- :-	最高	() 括号、字面量、变量		一元	-、!		乘法	* / %		加法	+ -		比较	== != < > <= >=		逻辑	&& \ \		最低	赋值（不在表达式中处理，由 <code>parse_stmt</code> 处理）	
-----	-----	---------	----	---------------	--	----	-----	--	----	-------	--	----	-----	--	----	-----------------	--	----	--------	--	----	---	--

5.5 经典的递归下降解析

```
// 入口：解析整个表达式
fn parse_expr(input: ParseStream) -> syn::Result<Expr> {
    parse_logical_or(input)
}

// expr || expr ...
fn parse_logical_or(input: ParseStream) -> syn::Result<Expr> {
    let mut left = parse_logical_and(input)?;
    while input.peek(Token![||]) {
        input.parse::()?;
        let right = parse_logical_and(input)?;
        left = Expr::BinaryOp {
            lhs: Box::new(left),
            op: BinaryOp::Or,
            rhs: Box::new(right),
        };
    }
    Ok(left)
}

// expr && expr ...
fn parse_logical_and(input: ParseStream) -> syn::Result<Expr> {
    let mut left = parse_comparison(input)?;
    while input.peek(Token![&&]) {
        input.parse::()?;
        let right = parse_comparison(input)?;
        left = Expr::BinaryOp {
            lhs: Box::new(left),
            op: BinaryOp::And,
            rhs: Box::new(right),
        };
    }
    Ok(left)
}

// expr == expr 等等
fn parse_comparison(input: ParseStream) -> syn::Result<Expr> {
    let mut left = parse_add_sub(input)?;
    if input.peek(Token![==]) {
        input.parse::()?;
        let right = parse_add_sub(input)?;
        return Ok(Expr::BinaryOp {
            lhs: Box::new(left), op: BinaryOp::Eq, rhs: Box::new(right),
        });
    }
    if input.peek(Token![!=]) {
```

```

        input.parse::<Token![!=]>()?;
        let right = parse_add_sub(input)?;
        return Ok(Expr::BinaryOp {
            lhs: Box::new(left), op: BinaryOp::Neq, rhs: Box::new(right),
        });
    }
    if input.peek(Token![<]) {
        input.parse::<Token![<]>()?;
        let right = parse_add_sub(input)?;
        return Ok(Expr::BinaryOp {
            lhs: Box::new(left), op: BinaryOp::Lt, rhs: Box::new(right),
        });
    }
    if input.peek(Token![>]) {
        input.parse::<Token![>]>()?;
        let right = parse_add_sub(input)?;
        return Ok(Expr::BinaryOp {
            lhs: Box::new(left), op: BinaryOp::Gt, rhs: Box::new(right),
        });
    }
    if input.peek(Token![<=]) {
        input.parse::<Token![<=]>()?;
        let right = parse_add_sub(input)?;
        return Ok(Expr::BinaryOp {
            lhs: Box::new(left), op: BinaryOp::Lteq, rhs: Box::new(right),
        });
    }
    if input.peek(Token![>=]) {
        input.parse::<Token![>=]>()?;
        let right = parse_add_sub(input)?;
        return Ok(Expr::BinaryOp {
            lhs: Box::new(left), op: BinaryOp::Gteq, rhs: Box::new(right),
        });
    }
    Ok(left)
}

// expr + expr, expr - expr
fn parse_add_sub(input: ParseStream) -> syn::Result<Expr> {
    let mut left = parse_mul_div(input)?;
    loop {
        if input.peek(Token![+]) {
            input.parse::<Token![+]>()?;
            let right = parse_mul_div(input)?;
            left = Expr::BinaryOp {
                lhs: Box::new(left), op: BinaryOp::Add, rhs: Box::new(right),
            };
        } else if input.peek(Token![ - ]) {
            input.parse::<Token![ - ]>()?;
            let right = parse_mul_div(input)?;
            left = Expr::BinaryOp {
                lhs: Box::new(left), op: BinaryOp::Sub, rhs: Box::new(right),
            };
        } else {
            break;
        }
    }
}

```

```

    }
  }
  Ok(left)
}

// expr * expr, expr / expr, expr % expr
fn parse_mul_div(input: ParseStream) -> syn::Result<Expr> {
  let mut left = parse_unary(input)?;
  loop {
    if input.peek(Token![*]) {
      input.parse::

```

```

    Ok(Expr::Number(num))
  } else if input.peek(syn::LitStr) {
    let lit: syn::LitStr = input.parse()?;
    Ok(Expr::String(lit.value()))
  } else if input.peek(Token![true]) {
    input.parse::<Token![true]>()?;
    Ok(Expr::Number(1.0))
  } else if input.peek(Token![false]) {
    input.parse::<Token![false]>()?;
    Ok(Expr::Number(0.0))
  } else if input.peek(syn::Ident) {
    let ident: syn::Ident = input.parse()?;
    Ok(Expr::Variable(ident))
  } else if input.peek(syn::token::Paren) {
    let content;
    syn::parenthesized!(content in input);
    parse_expr(&content)
  } else {
    Err(input.error("expected expression"))
  }
}

```

5.6 特殊语句解析

除了标准控制流，还需要解析 DSL 特有的“函数调用”：

5.6.1 print / print_flush

```

fn parse_special_call(input: ParseStream) -> syn::Result<Option<Stmt>> {
  let ident: Ident = input.parse()?;
  match ident.to_string().as_str() {
    "print" => {
      let content;
      syn::parenthesized!(content in input);
      let expr = parse_expr(&content)?;
      input.parse::<Token![;]>()?;
      Ok(Some(Stmt::Print(expr)))
    }
    "print_flush" => {
      let content;
      syn::parenthesized!(content in input);
      let block: Ident = content.parse()?;
      input.parse::<Token![;]>()?;
      Ok(Some(Stmt::PrintFlush(block)))
    }
    "sensor" => {
      let content;
      syn::parenthesized!(content in input);
      let object: Ident = content.parse()?;
      content.parse::<Token![,]>()?;
      let property: Ident = content.parse()?;
      input.parse::<Token![;]>()?;
      Ok(Some(Stmt::Sensor { result: ident, object, property }))
    }
    "enable" | "disable" => {
      // ...
    }
  }
}

```

```
    }  
    _ => Ok(None),  
  }  
}
```

5.7 解析器的位置

解析器完整代码约 300-400 行。关键技巧：



概念

1. **递归下降**：每层函数处理一个优先级级别
2. **peek 先行**：看下个 token 但不消费，决定走哪个分支
3. **错误传播**：用 `syn::Result` 和 `?`，失败时自动带 `span` 信息
4. **syn 辅助**：`braced!`、`parenthesized!` 等宏简化括号处理

5.8 小结

- 手写解析器因为 DSL 不是标准 Rust 语法
- 递归下降法：每个优先级一层函数调用
- peek/parse 模式：`input.peek(...)` 先行，`input.parse()` 消费
- AST 保持嵌套结构——表达式树尚未展开
- syn 的 `ParseStream` 是通用 token 流解析器，不限于 Rust 语法
- 整个解析器约 300-400 行就能处理我们的 DSL 全集

6 中间表示 — 三地址码

🔧 实验记录：第五站：从 AST 到三地址码

AST 中的表达式可以是任意深度的树—— $a + b * (c - d)$ 在 AST 中是一个嵌套的 BinaryOp 结构。但 MLOG 的 op 指令一次只能做一次运算。

中间表示 (IR) 的作用就是把 AST 树拍平成 MLOG 能消费的二地址码序列。

6.1 什么是三地址码

定义

三地址码 (Three-Address Code, TAC) 是一种中间表示，每条指令最多有三个操作数：

```
result = operand1 操作符 operand2
```

这恰好对应 MLOG 的 op 指令格式。

复杂表达式 $a + b * c$ 分解为：

```
t0 = b * c
t1 = a + t0
```

6.2 IR 指令定义

```
// ir.rs
pub enum IrInstr {
    // set result value
    Set {
        result: String,
        value: IrValue,
    },

    // op <opcode> result lhs rhs
    Op {
        opcode: String,
        result: String,
        lhs: IrValue,
        rhs: IrValue,
    },

    // jump label (always)
    Jump {
        label: String,
    },

    // jump label condition lhs rhs
    JumpIf {
        label: String,
        condition: String, // "eq", "lt", "gt", "not", ...
        lhs: IrValue,
        rhs: IrValue,
    },

    // :label
    Label(String),
}
```

```

// 特殊指令
Print(IrValue),
PrintFlush(String),
Sensor {
    result: String,
    object: String,
    property: String,
},
Control {
    action: String, // "enabled", "shoot", ...
    block: String,
    value: IrValue,
},
Ubind(String),
// ...
}

pub enum IrValue {
    Number(f64),
    String(String),
    Var(String), // 已命名的变量
    Temp(usize), // 临时变量 __tmp_0, __tmp_1...
    Special(String), // @unit, @time 等
}

// 整个程序的 IR
pub struct IrProgram {
    pub instrs: Vec<IrInstr>,
    pub temp_counter: usize,
}

```

6.3 将 AST 表达式降级为三地址码

核心挑战：将一棵嵌套的表达式树“拍平”成一条条三地址码指令。

算法思路：

- 每次遇到 BinaryOp { lhs, op, rhs }, 先递归处理 lhs 和 rhs, 分别得到它们对应的“结果变量”
- 然后分配一个新临时变量, 生成一条 op 指令
- 返回这个临时变量的值

```

impl IrProgram {
    fn lower_expr(&mut self, expr: &Expr) -> IrValue {
        match expr {
            Expr::Number(n) => IrValue::Number(*n),
            Expr::String(s) => IrValue::String(s.clone()),
            Expr::Variable(ident) => IrValue::Var(ident.to_string()),

            Expr::BinaryOp { lhs, op, rhs } => {
                let lhs_val = self.lower_expr(lhs);
                let rhs_val = self.lower_expr(rhs);
                let result = self.new_temp();

                let opcode = match op {
                    BinaryOp::Add => "add",

```

```

        BinaryOp::Sub => "sub",
        BinaryOp::Mul => "mul",
        BinaryOp::Div => "div",
        BinaryOp::Mod => "mod",
        BinaryOp::Eq  => "eq",
        BinaryOp::Neq => "neq",
        BinaryOp::Lt  => "lt",
        BinaryOp::Gt  => "gt",
        BinaryOp::Lteq => "lteq",
        BinaryOp::Gteq => "gteq",
        BinaryOp::And  => "land",
        BinaryOp::Or   => "lor", // 注意：MLOG 逻辑或用位或
    };

    self.instrs.push(IrInstr::Op {
        opcode: opcode.to_string(),
        result: result.clone(),
        lhs: lhs_val,
        rhs: rhs_val,
    });

    IrValue::Var(result)
}

Expr::UnaryOp { op, expr } => {
    let val = self.lower_expr(expr);
    match op {
        UnaryOp::Neg => {
            let result = self.new_temp();
            // -x = 0 - x
            self.instrs.push(IrInstr::Op {
                opcode: "sub".to_string(),
                result: result.clone(),
                lhs: IrValue::Number(0.0),
                rhs: val,
            });
            IrValue::Var(result)
        }
        UnaryOp::Not => {
            let result = self.new_temp();
            // !x = x == false
            self.instrs.push(IrInstr::Op {
                opcode: "eq".to_string(),
                result: result.clone(),
                lhs: val,
                rhs: IrValue::Number(0.0),
            });
            IrValue::Var(result)
        }
    }
}

}

}

}

fn new_temp(&mut self) -> String {

```

```

    let id = self.temp_counter;
    self.temp_counter += 1;
    format!("__tmp_{}", id)
  }
}

```

6.4 示例：表达式分解

输入 AST (a + b * c):

```

BinaryOp {
  lhs: Variable("a"),
  op: Add,
  rhs: BinaryOp {
    lhs: Variable("b"),
    op: Mul,
    rhs: Variable("c"),
  },
}

```

lower_expr 的递归过程:

```

lower_expr(BinaryOp(Add))
├─ lower_expr(Variable("a")) → Var("a")
└─ lower_expr(BinaryOp(Mul))
    ├─ lower_expr(Variable("b")) → Var("b")
    └─ lower_expr(Variable("c")) → Var("c")
    分配临时变量 __tmp_0
    生成: op mul __tmp_0 b c
    返回 Var("__tmp_0")

    分配临时变量 __tmp_1
    生成: op add __tmp_1 a __tmp_0
    返回 Var("__tmp_1")

```

最终 IR:

```

op mul __tmp_0 b c
op add __tmp_1 a __tmp_0

```

6.5 将语句降级为 IR

```

impl IrProgram {
  fn lower_stmt(&mut self, stmt: &Stmt, labels: &mut LabelState) {
    match stmt {
      Stmt::Let { name, init, .. } => {
        let val = self.lower_expr(init);
        self.instrs.push(IrInstr::Set {
          result: name.to_string(),
          value: val,
        });
      }
    }
  }
}

```

```

Stmt::Assign { name, value } => {
    let val = self.lower_expr(value);
    self.instrs.push(IrInstr::Set {
        result: name.to_string(),
        value: val,
    });
}

Stmt::If { condition, then_branch, else_branch } => {
    let cond_val = self.lower_expr(condition);
    let skip_label = labels.fresh("__skip");
    let else_label = else_branch.as_ref()
        .map(|_| labels.fresh("__else"));
    let end_label = labels.fresh("__end_if");

    // 条件跳转
    if let Some(ref el) = else_label {
        self.instrs.push(IrInstr::JumpIf {
            label: el.clone(),
            condition: "eq".to_string(),
            lhs: cond_val,
            rhs: IrValue::Number(0.0),
        });
    } else {
        self.instrs.push(IrInstr::JumpIf {
            label: skip_label.clone(),
            condition: "eq".to_string(),
            lhs: cond_val,
            rhs: IrValue::Number(0.0),
        });
    }

    // then 分支
    for s in then_branch {
        self.lower_stmt(s, labels);
    }

    if let Some(el) = else_label {
        self.instrs.push(IrInstr::Jump {
            label: end_label.clone(),
        });
        self.instrs.push(IrInstr::Label(el));
        for s in el {
            self.lower_stmt(s, labels);
        }
    }

    self.instrs.push(IrInstr::Label(
        else_label.unwrap_or(skip_label)
    ));
}

Stmt::While { condition, body } => {
    let loop_label = labels.fresh("__while");
    let end_label = labels.fresh("__end_while");

```

```

        self.instrs.push(IrInstr::Label(loop_label.clone()));

        let cond_val = self.lower_expr(condition);
        // if condition == false → jump to end
        self.instrs.push(IrInstr::JumpIf {
            label: end_label.clone(),
            condition: "eq".to_string(),
            lhs: cond_val,
            rhs: IrValue::Number(0.0),
        });

        for s in body {
            self.lower_stmt(s, labels);
        }

        self.instrs.push(IrInstr::Jump {
            label: loop_label,
        });
        self.instrs.push(IrInstr::Label(end_label));
    }

    Stmt::Loop { body } => {
        let loop_label = labels.fresh("__loop");
        let end_label = labels.fresh("__break");

        labels.push_break(end_label.clone());
        self.instrs.push(IrInstr::Label(loop_label.clone()));

        for s in body {
            self.lower_stmt(s, labels);
        }

        self.instrs.push(IrInstr::Jump {
            label: loop_label,
        });
        self.instrs.push(IrInstr::Label(end_label));
        labels.pop_break();
    }

    Stmt::Break => {
        let break_label = labels.current_break()
            .expect("break outside loop");
        self.instrs.push(IrInstr::Jump {
            label: break_label,
        });
    }

    Stmt::Print(expr) => {
        let val = self.lower_expr(expr);
        self.instrs.push(IrInstr::Print(val));
    }

    Stmt::PrintFlush(block) => {
        self.instrs.push(IrInstr::PrintFlush(block.to_string()));
    }

```

```

    }

    Stmt::ExprStmt(expr) => {
        self.lower_expr(expr); // 结果被丢弃
    }

    // ... sensor, enable, disable 等
}
}
}

```

6.6 标签管理

标签在控制流中很重要——每个 `if/while/loop` 都需要唯一的标签名。

```

struct LabelState {
    counter: usize,
    break_stack: Vec<String>,
}

impl LabelState {
    fn fresh(&mut self, prefix: &str) -> String {
        let id = self.counter;
        self.counter += 1;
        format!("{}", prefix, id)
    }

    fn push_break(&mut self, label: String) {
        self.break_stack.push(label);
    }

    fn pop_break(&mut self) {
        self.break_stack.pop();
    }

    fn current_break(&self) -> Option<String> {
        self.break_stack.last().cloned()
    }
}

```

🔍 核心直觉

IR 阶段做了两件事：

1. **表达式展平**：将 AST 的嵌套表达式树变成三地址码序列
2. **控制流结构化**：将 `if/while/loop` 转成标签 + 跳转

经过 IR 后，指令序列已经非常接近 MLOG——剩下的只是把每个 IR 指令映射到 MLOG 文本。

6.7 小结

- 三地址码是 MLOG 的自然中间表示
- `lower_expr` 递归遍历 AST 表达式树，分配临时变量，生成 `op` 指令
- `lower_stmt` 处理控制流，生成标签 + `jump` 指令
- 标签管理器 (LabelState) 确保标签名唯一，`break` 知道跳到哪里
- 经过 IR 阶段，编译器已经“说 MLOG 了”

7 代码生成 – IR → MLOG 文本

 实验记录：第六站：生成指令

IR 已经是 MLOG 的近亲。现在把每个 IR 指令一对一地映射到 MLOG 文本。

7.1 IrInstr → MLOG 字符串

```
// codegen.rs
pub fn generate_milog(instrs: &[IrInstr]) -> String {
    let mut out = String::new();

    for instr in instrs {
        let line = match instr {
            IrInstr::Set { result, value } => {
                format!("set {} {}", result, fmt_value(value))
            }

            IrInstr::Op { opcode, result, lhs, rhs } => {
                format!("op {} {} {} {}",
                    opcode, result,
                    fmt_value(lhs),
                    fmt_value(rhs),
                )
            }

            IrInstr::Jump { label } => {
                format!("jump {} always", label)
            }

            IrInstr::JumpIf { label, condition, lhs, rhs } => {
                let cond_str = match condition.as_str() {
                    "eq" => "equal",
                    "neq" => "notEqual",
                    "lt" => "lessThan",
                    "gt" => "greaterThan",
                    "lteq" => "lessThanEq",
                    "gteq" => "greaterThanEq",
                    "not" => "not",
                    _ => condition,
                };
                format!("jump {} {} {} {} {}",
                    label, cond_str,
                    fmt_value(lhs),
                    fmt_value(rhs),
                )
            }

            IrInstr::Label(name) => {
                format!("{:}", name)
            }

            IrInstr::Print(val) => {
                format!("print {}", fmt_value(val))
            }

            IrInstr::PrintFlush(block) => {
```



```

        format!("printflush {}", block)
    }

    IrInstr::Sensor { result, object, property } => {
        format!("sensor {} {} {}", result, object, property)
    }

    IrInstr::Control { action, block, value } => {
        format!("control {} {} {} 0 0 0",
            action, block, fmt_value(value))
    }

    IrInstr::Ubind(unit_type) => {
        format!("ubind {}", unit_type)
    }
    // ...
};

out.push_str(&line);
out.push('\n');
}

out
}

```

7.2 值的格式化

```

fn fmt_value(val: &IrValue) -> String {
    match val {
        IrValue::Number(n) => {
            // MLOG 中的数字直接用标准格式
            if *n == n.floor() && n.is_finite() {
                format!("{}", *n as i64)
            } else {
                format!("{}", n)
            }
        }
        IrValue::String(s) => {
            // MLOG 字符串用双引号
            format!("\"{}\"", s)
        }
        IrValue::Var(name) => name.clone(),
        IrValue::Temp(id) => format!("__tmp_{}", id),
        IrValue::Special(name) => name.clone(),
    }
}

```

概念

`fmt_value` 是代码生成中最关键的小函数。

它把 IR 中的抽象值类型转换为 MLOG 文本中的具体表示。注意：

- 整数去掉 `.0` 后缀 (MLOG 中 `5` 比 `5.0` 更自然)
- 字符串加双引号
- 临时变量名按 `__tmp_N` 格式

7.3 完整的代码生成示例

输入 IR:

```
set i 0
set limit 5
set sum 0
:__while_0
op lt __tmp_0 i limit
jump __end_while_0 equal __tmp_0 false
op add sum sum i
op add i i 1
jump __while_0 always
:__end_while_0
print "Sum: "
print sum
printf flush message1
```

输出 MLOG (和 IR 一模一样——因为 IR 已经设计为 1:1 映射):

```
set i 0
set limit 5
set sum 0
:__while_0
op lt __tmp_0 i limit
jump __end_while_0 equal __tmp_0 false
op add sum sum i
op add i i 1
jump __while_0 always
:__end_while_0
print "Sum: "
print sum
printf flush message1
```

7.4 MLOG 变量名的考虑

MLOG 变量名有一些隐含规则:

1. 区分大小写: counter 和 Counter 是不同的
2. 不能以数字开头
3. 不能包含空格或特殊字符
4. @ 开头的是内建变量
5. 不能用 MLOG 关键字: set op jump read write 等

我们的编译器做几件事来保证安全:

- 临时变量以 __tmp_ 开头——用户不太可能重名
- 标签以 __ 前缀确保不会冲突
- 检查用户变量名不冲突 (诊断阶段)

7.5 一个值得注意的设计选择

为什么不把 IR 省掉、直接从 AST 生成 MLOG?

AST → MLOG (略过 IR)

🔍 核心直觉

没有 IR 阶段也能工作——AST 语句的代码生成可以内联处理表达式。

但有 IR 的好处：

1. **清晰的分界**：AST 阶段关心“语法”，IR 阶段关心“指令序列”，codegen 阶段关心“文本格式”
2. **可测试**：每个阶段的输出都可以独立验证
3. **可优化**：未来可以在 IR 层面做优化（消除冗余临时变量、合并相邻指令）
4. **可调试**：出问题，打印 IR 中间状态非常有用

对于教学目的，三层架构清晰、易理解。对于生产环境，两层（AST → MLOG）也可以工作。

7.6 在宏入口中串联

回到 lib.rs，串联所有阶段：

```
#[proc_macro]
pub fn mlog(input: TokenStream) -> TokenStream {
    let input: proc_macro2::TokenStream = input.into();

    // Phase 1: Parse
    let program = match parser::parse_program(input) {
        Ok(ast) => ast,
        Err(err) => return err.to_compile_error().into(),
    };

    // Phase 2: Lower to IR
    let ir = ir::lower_program(&program);

    // Phase 3: Generate MLOG text
    let mlog_text = codegen::generate_mlog(&ir.instrs);

    // Phase 4: Return as string literal
    quote::quote! { #mlog_text }.into()
}
```

7.7 小结

- 代码生成是 IR → MLOG 文本的直接转换
- `fmt_value` 处理值的 MLOG 表示（数字去 `.0`、字符串加引号）
- IR 已经设计为 1:1 映射，代码生成几乎是逐条翻译
- 编译器管好变量名——`__tmp_N` 避免冲突
- 三层架构（AST → IR → MLOG）清晰、可测试
- 四个阶段在宏入口中串联，编译时完成全部工作

8 控制流 – jump 的三种形态

 实验记录：第七站：if / while / loop / break

MLOG 只有一种控制流原语——jump。所有高级控制结构（分支、循环、break）都必须转化为标签 + 跳转的组合。

这一章深入讲解每种控制结构的 jump 模式。

8.1 if-else 的 jump 模式

8.1.1 只有 if (无 else)

```
if cond {  
    // body  
}
```

IR 模式：

```
<计算 cond → tmp>  
jump __skip equal tmp false    // if cond == false, skip body  
<body>  
:__skip
```

MLOG：

```
op lt __tmp_0 i 10  
jump __skip_0 equal __tmp_0 false  
print "less than 10"  
:__skip_0
```

8.1.2 if-else

```
if cond {  
    // then_body  
} else {  
    // else_body  
}
```

IR 模式：

```
<计算 cond → tmp>  
jump __else equal tmp false    // if cond == false, goto else  
<then_body>  
jump __end always  
:__else  
<else_body>  
:__end
```



概念

关键点：then 分支末尾必须有 jump __end always，否则会落到 else 分支中。

MLOG 是顺序执行的——没有“自动跳过 else”的机制。每个分支末尾的 jump 都不可或缺。

8.1.3 else if

```
if a > 0 {  
    // a为正  
} else if a < 0 {  
    // a为负  
} else {  
    // a为零  
}
```

IR 模式:

```
<计算 a > 0 → tmp0>  
jump __else_if equal tmp0 false  
<正>  
jump __end always  
:__else_if  
<计算 a < 0 → tmp1>  
jump __else equal tmp1 false  
<负>  
jump __end always  
:__else  
<零>  
:__end
```

8.2 while 循环的 jump 模式

```
while cond {  
    // body  
}
```

IR 模式:

```
::__while_N  
<计算 cond → tmp>  
jump __end_while_N equal tmp false    // if !cond, exit  
<body>  
jump __while_N always                // loop back  
:__end_while_N
```

核心直觉

while 循环的标签布局:

```
:loop_head      ← 每次迭代开始  
condition check  
conditional jump to :loop_end if false  
body  
unconditional jump back to :loop_head  
:loop_end       ← 退出点
```

和汇编/字节码中的 while 结构完全一致。MLOG 是一个比较友好的编译目标—— jump 指令的条件语法比真正的汇编更接近高级语言。

8.3 loop 循环 + break

```
loop {  
    if exit_cond { break; }  
    // body  
}
```

IR 模式:

```
:__loop_N  
<计算 exit_cond → tmp>  
jump __break_N not equal tmp false    // if exit_cond, goto break  
<body>  
jump __loop_N always  
:__break_N
```

break 的实现最有趣: 解析到 break 时, 我们查看 break_stack 找到最近的 break 标签名, 直接生成一条 jump __break_N。

概念

因为 loop 需要知道 break 跳到哪里, 所以:

- 进入 loop 时, 把一个新标签 __break_N 压入堆栈
- 遇到 break 时, 从堆栈读取当前标签
- 离开 loop 时, 弹出堆栈

嵌套 loop 中, 每个 break 自动跳转到对应的层级。

```
loop {           // break → __break_0  
    loop {       // break → __break_1  
        break;   // 跳到 __break_1  
    }  
    break;       // 跳到 __break_0  
}
```

对应 IR:

```
:__loop_0  
  :__loop_1  
    jump __break_1 always    // 内层 break  
    jump __loop_1 always  
  :__break_1  
    jump __break_0 always    // 外层 break  
    jump __loop_0 always  
  :__break_0
```

8.4 短路求值

&& 和 || 需要短路语义——不计算不需要的操作数。

```
// a && b: 如果 a 是 false, 不计算 b
```

IR:

```

<计算 a → tmp_a>
jump __skip_and equal tmp_a false      // a == false? skip
<计算 b → tmp_b>
set __result tmp_b
jump __end_and
: __skip_and
set __result false
: __end_and

```

```
// a || b: 如果 a 是 true, 不计算 b
```

IR:

```

<计算 a → tmp_a>
jump __skip_or notEqual tmp_a false    // a == true? skip
<计算 b → tmp_b>
set __result tmp_b
jump __end_or
: __skip_or
set __result true
: __end_or

```

核心直觉

短路求值就是提前跳转。

&& 在第一个为 false 时跳转到“整体结果为 false”的标签。|| 在第一个为 true 时跳转到“整体结果为 true”的标签。

注意 MLOG 的 jump 条件: jump label condition a b 意思是“如果 a condition b 为真则跳转”。

- “a == false 时跳转” → jump label equal a false
- “a == true 时跳转” → jump label notEqual a false

8.5 条件跳转的条件码速查

比较	MLOG 条件	语义	:- :- :-	a == b	equal	a 等于 b	a != b
notEqual	a 不等于 b	a < b	lessThan	a 小于 b	a <= b	lessThanEq	a 小于等于 b
a > b	greaterThan	a 大于 b	a >= b	greaterThanEq	a 大于等于 b	无条件	always
永远跳转							

条件码与 op 的运算码使用了不同的名称——代码生成时需要注意转换。

8.6 小结

- MLOG 只有 jump 一种控制流——所有结构化控制流必须转化
- if 无 else: jump skip equal cond false
- if-else: jump else equal cond false + jump end always
- while: label loop + jump end equal cond false + jump loop always
- loop/break: break_stack 管理标签层级
- 短路求值 = 提前跳转到结果标签
- 每个分支末尾的 jump always 不能少——MLOG 没有「跳过 else」

9 变量系统

✍ 实验记录： 第八站：作用域与唯一名称

MLOG 变量是全局的。DSL 中我们写了 `let x = 5;`，在 MLOG 中它就是 `set x 5`。

但如果有作用域呢？如果两个嵌套的 `block` 中有同名变量呢？

9.1 MLOG 的变量模型

定义

在 MLOG 中：

- 变量由 `set` 指令在运行时创建
- 一个处理器中所有变量是全局可见的
- 没有作用域的概念
- 变量名就是字面名字——`counter` 在整个程序生命周期内指向同一个值

编译器要解决的问题：将 DSL 中的语义作用域映射到 MLOG 的平面命名空间。

9.2 问题场景

```
let x = 10;
if something {
    let x = 20; // 新作用域中的 x
    print(x);   // 应该输出 20
}
print(x);      // 应该输出 10 (外层 x 没有被改变)
```

如果直接把两个 `x` 都翻译为 MLOG 的 `set x`，那么内层会覆盖外层——语义错误。

9.3 解决方案一：扁平化

对于简单的 DSL，可以禁止 `shadowing`——同一个变量只能声明一次：

```
let x = 10;
if something {
    let x = 20; // 编译错误：'x' already defined
}
```

这最简单，但用户体验不好——不能在不同作用域中复用变量名。

9.4 解决方案二：唯一名称

更优雅的方案——给每个 `let` 声明的变量分配唯一的内部名称：

```
// DSL
let x = 10;
if something {
    let x = 20;
    print(x);
}
print(x);
```

内部映射：


```
x_0 = 10
x_1 = 20
print(x_1)
print(x_0)
```

MLOG:

```
set x_0 10
op neq __tmp_0 something 0
jump __skip equal __tmp_0 false
set x_1 20
print x_1
:__skip
print x_0
```

用户只看到 `x`，但编译器背后用 `x_0`、`x_1` 等唯一名称。

9.5 作用域管理

```
// vars.rs - 作用域 + 唯一名称管理
use std::collections::HashMap;

pub struct ScopeManager {
    // 当前作用域层级 (0 = 全局)
    depth: usize,
    // 变量名 -> 唯一 MLOG 名
    names: HashMap<String, String>,
    // 计数器，确保唯一
    counter: u64,
}

impl ScopeManager {
    pub fn new() -> Self {
        ScopeManager {
            depth: 0,
            names: HashMap::new(),
            counter: 0,
        }
    }

    // 进入新作用域 (if body, while body, loop body)
    pub fn enter(&mut self) {
        self.depth += 1;
    }

    // 离开作用域
    pub fn leave(&mut self) {
        self.depth -= 1;
    }

    // 声明新变量
    pub fn declare(&mut self, name: &str) -> String {
        let unique = format!("__{}_{}", name, self.counter);
        self.counter += 1;
    }
}
```

```

        self.names.insert(name.to_string(), unique.clone());
        unique
    }

    // 查找变量的唯一名称
    pub fn lookup(&self, name: &str) -> Option<&str> {
        self.names.get(name).map(|s| s.as_str())
    }
}

```

9.6 临时变量

临时变量不需要作用域管理——它们在 MLOG 中是全局的，但名称唯一即可。

```

// 临时变量生成
fn new_temp(&mut self) -> String {
    let id = self.temp_counter;
    self.temp_counter += 1;
    format!("__t{}", id)
}

```

临时变量在 MLOG 中生成 __t0、__t1.....和用户变量区分开。

概念

变量命名的层次策略：

前缀	来源	示例	:- :- :-	无前缀	用户声明的变量（唯一化后）	__counter_0
	__t	编译器自动生成的临时变量	__t0		标签	__while_0, __break_1
@	MLOG 内建变量	@unit, @time				

用户不可能写出以 __ 开头的变量名（诊断阶段会报错），所以不会有冲突。

9.7 不可变性检查

let vs let mut 的语义差异在 DSL 解析时检查：

```

// 解析 let 语句时记录可变性
struct VarInfo {
    unique_name: String,
    mutable: bool,
}

// 解析赋值语句时检查
fn check_assign(name: &str) -> Result<(), DiagError> {
    if let Some(info) = scope.lookup_info(name) {
        if !info.mutable {
            return Err(DiagError::new(
                span,
                format!("cannot assign to immutable variable '{}'", name),
            ));
        }
    }
    Ok(())
}

```

如果用户写了 `let x = 5;` 后又 `x = 10;`, 编译器在编译时拒绝——因为 `let` 变量不可变。

9.8 完整例子

DSL:

```
let x = 10;           // → __x_0
let mut y = 5;        // → __y_1
y = y + 1;            // → set __y_1 (op add __t0 __y_1 1)
if y > 5 {
    let x = 20;        // → __x_2 (新变量, 不覆盖 __x_0)
    print(x);          // → print __x_2
}
print(x);             // → print __x_0
```

MLOG (带唯一名称):

```
set __x_0 10
set __y_1 5
op add __t0 __y_1 1
set __y_1 __t0
op gt __t1 __y_1 5
jump __skip_0 equal __t1 false
set __x_2 20
print __x_2
: __skip_0
print __x_0
```

用户看到的 DSL 是干净的, 编译器管理的 MLOG 是唯一的——两全其美。

9.9 小结

- MLOG 没有作用域——编译器必须管理
- 唯一名称化: 每个 `let` 声明分配全局唯一的名字
- 作用域管理器跟踪进入/离开作用域
- 临时变量用独立命名空间 (`__tN`)
- 用户变量以 `__` 开头 + 计数器后缀防止冲突
- 不可变性在编译时检查——不产生 MLOG 指令
- 最终 MLOG 中的变量名是自动生成的, 用户无需关心

10 输出生成 — 最后的 MLOG 文本

✍ 实验记录： 第九站：格式化与集成

MLOG 指令序列已经有了。现在要把它变成用户可以使用形式——一个 `&str` 字符串字面量。

10.1 生成最终的字符串

```
// codegen.rs
pub fn generate_mlog(instrs: &[IrInstr]) -> String {
    let mut out = String::new();
    for instr in instrs {
        match instr {
            IrInstr::Set { result, value } => {
                out.push_str(&format!("set {} {}\n", result, fmt_value(value)));
            }
            IrInstr::Op { opcode, result, lhs, rhs } => {
                out.push_str(&format!("op {} {} {} {}\n",
                    opcode, result, fmt_value(lhs), fmt_value(rhs)));
            }
            IrInstr::Jump { label } => {
                out.push_str(&format!("jump {} always\n", label));
            }
            IrInstr::JumpIf { label, condition, lhs, rhs } => {
                out.push_str(&format!("jump {} {} {} {}\n",
                    label, condition, fmt_value(lhs), fmt_value(rhs)));
            }
            IrInstr::Label(name) => {
                out.push_str(&format!(":{}\n", name));
            }
            IrInstr::Print(val) => {
                out.push_str(&format!("print {}\n", fmt_value(val)));
            }
            IrInstr::PrintFlush(block) => {
                out.push_str(&format!("printflush {}\n", block));
            }
            // ... 其他指令
        }
    }
    out
}
```

输出是纯文本 MLOG——每个指令一行。

10.2 可选：添加注释

```
// 可选：在生成的 MLOG 中添加注释
fn generate_with_comments(ir: &IrProgram, source: &str) -> String {
    let mut out = String::new();
    out.push_str("# Generated by rust2mlog\n");
    out.push_str(&format!("# Source:\n"));
    for line in source.lines() {
        out.push_str(&format!("#   {}\n", line));
    }
    out.push('\n');
    out.push_str(&generate_mlog(&ir.instrs));
}
```

```
    out
}
```

MLOG 中 # 开头的行是注释。添加注释有助于调试——可以看到原始 DSL 和生成的 MLOG 的对应关系。

10.3 回到 macro 入口

最终的宏入口将 MLOG 文本返回为 Rust 字符串字面量：

```
#[proc_macro]
pub fn mlog(input: TokenStream) -> TokenStream {
    let input_ts: proc_macro2::TokenStream = input.into();

    let program = match parser::parse_program(input_ts.clone()) {
        Ok(ast) => ast,
        Err(err) => return err.to_compile_error().into(),
    };

    let ir = ir::lower_program(&program);
    let mlog_text = codegen::generate_mlog(&ir.instrs);

    // 关键：把 MLOG 文本包裹为 Rust 字符串字面量
    let output = quote::quote! { #mlog_text };
    output.into()
}
```

概念

quote! { #mlog_text } 中 mlog_text 是 String。quote 会把它展开为字符串字面量。

```
// 宏展开前
let program = mlog! { set x 5 };

// 宏展开后（由 proc macro 生成）
let program = "set x 5\n";
```

program 的类型是 &'static str——编译时确定，零运行时开销。

10.4 用户如何使用生成的 MLOG

10.4.1 直接嵌入

```
const MY_PROGRAM: &str = mlog! {
    let mut i = 0;
    while i < 10 {
        print(i);
        i += 1;
    }
};

fn main() {
    println!("MLOG program:\n{}", MY_PROGRAM);
    // 复制到 Mindustry 处理器
}
```

10.4.2 写入文件

```
use std::fs;

fn main() {
    let mlog = mlog! {
        ubind(@poly);
        ucontrol move 100 200 0 0 0;
    };
    fs::write("my_program.mlog", mlog).unwrap();
}
```

10.4.3 配合 build.rs 生成输出文件

在 build.rs 中不能直接用 proc macro, 但可以:

```
// build.rs - 生成包含 MLOG 的常量
fn main() {
    // 生成一个 .rs 文件, 其中包含 MLOG 常量
    let mlog_code = include_str!("my_program.mlog");
    // ...
}
```

10.5 格式化细节

MLOG 对格式不敏感——最重要的是换行。但好的格式有助于调试:

```
# 好的格式
set x 5
op add x x 1
jump loop equal x 10

# 也能工作, 但调试麻烦
set x 5
op add x x 1
jump loop equal x 10
```

我们选择简洁格式——每条指令一行, 无多余缩进。标签独占一行。

10.6 小结

- 代码生成的最终输出是 String
- 宏把 String 展开为 &'static str 字面量
- 可选添加注释以帮助调试
- 用户可以用 const、文件写入等方式使用生成的 MLOG
- MLOG 文本是纯文本——可以被任何工具链消费
- 编译时生成 = 零运行时开销

11 错误处理与诊断

🔧 实验记录：第十站：友好的报错

编译器不是只对“正确输入”负责——对“错误输入”的反馈质量同样重要。

📖 概念

过程宏中的错误处理目标：

1. 精确指向 DSL 源代码中的错误位置（用 `Span`）
2. 给出人类可读的错误信息（不是 `token dump`）
3. 尽量一次报告多个错误（而不止第一个）

11.1 `Span` 从哪来

`syn` 的 `Ident`、`Literal`、`Token` 都自带 `Span`——它们知道自己在原始输入中的位置。

```
// 解析时，每个 syn 节点都带了 span 信息
let ident: syn::Ident = input.parse()?;
// ident.span() 指向这个标识符在源文件中的行列位置
```

当解析失败时，直接用节点的 `span` 创建错误：

```
fn parse_let_stmt(input: ParseStream) -> syn::Result<Stmt> {
    input.parse::<Token![let]>()?;
    let name: Ident = input.parse()
        .map_err(|e| {
            // 保留原始 span，但改善错误信息
            syn::Error::new(e.span(),
                "expected variable name after 'let'")
        })?;
    // ...
}
```

11.2 语义错误 vs 语法错误

语法错误（`token` 不匹配）`syn` 自动处理。语义错误需要手动实现：

错误	类型	示例	语义
<code>let x = y; y 未定义</code>	语义	<code>error: variable 'y' is not defined</code>	
<code>break; 不在循环中</code>	语义	<code>error: 'break' outside loop</code>	
<code>x = 10; x 不可变</code>	语义	<code>error: cannot assign to immutable 'x'</code>	
<code>5 + "str"</code>	语义	<code>error: type mismatch</code>	

11.3 诊断系统

```
// diagnostics.rs
use proc_macro2::Span;
use syn::Error;

pub struct Diagnostic {
    errors: Vec<Error>,
}

impl Diagnostic {
    pub fn new() -> Self {
        Diagnostic { errors: Vec::new() }
    }
}
```

```

    }

    pub fn error(&mut self, span: Span, msg: impl Into<String>) {
        self.errors.push(Error::new(span, msg.into()));
    }

    pub fn emit(self) -> Result<(), Error> {
        if self.errors.is_empty() {
            return Ok(());
        }
        let mut combined = self.errors.into_iter()
            .reduce(|mut a, b| { a.combine(b); a })
            .unwrap();
        Err(combined)
    }
}

```

🔍 核心直觉

Diagnostic 收集多个错误，然后一次性发出。

`syn::Error` 的 `combine` 方法将多个错误合并为一个——编译器会显示所有错误位置。

用户可以在修复一轮后重新编译，看到下一组错误——就像 Rust 编译器的行为。

11.4 典型语义检查

11.4.1 未定义变量

```

fn check_variables(ast: &Program, diag: &mut Diagnostic) {
    let mut scope = ScopeManager::new();

    for stmt in &ast.stmts {
        check_stmt_vars(stmt, &mut scope, diag);
    }
}

fn check_stmt_vars(stmt: &Stmt, scope: &mut ScopeManager, diag: &mut Diagnostic)
{
    match stmt {
        Stmt::Let { name, init, .. } => {
            check_expr_vars(init, scope, diag);
            scope.declare(&name.to_string(), name.span());
        }
        Stmt::Assign { name, value } => {
            if !scope.contains(&name.to_string()) {
                diag.error(name.span(),
                    format!("variable '{}' is not defined", name));
            }
            if !scope.is_mutable(&name.to_string()) {
                diag.error(name.span(),
                    format!("cannot assign to immutable variable '{}'", name));
            }
            check_expr_vars(value, scope, diag);
        }
        // ... 其他语句
    }
}

```



```

    }
}

fn check_expr_vars(expr: &Expr, scope: &ScopeManager, diag: &mut Diagnostic) {
    if let Expr::Variable(ident) = expr {
        if !scope.contains(&ident.to_string()) {
            diag.error(ident.span(),
                format!("variable '{}' is not defined", ident));
        }
    }
    // ... 递归检查子表达式
}

```

11.4.2 break 不在循环中

```

fn check_break(stmts: &[Stmt], in_loop: bool, diag: &mut Diagnostic) {
    for stmt in stmts {
        match stmt {
            Stmt::Break => {
                if !in_loop {
                    diag.error(span_of(stmt),
                        "'break' can only be used inside a loop");
                }
            }
            Stmt::While { body, .. } | Stmt::Loop { body } => {
                check_break(body, true, diag);
            }
            _ => {}
        }
    }
}

```

11.5 与 syn 的错误处理集成

所有诊断检查在解析完成后、生成 IR 之前运行：

```

#[proc_macro]
pub fn mlog(input: TokenStream) -> TokenStream {
    let input_ts: proc_macro2::TokenStream = input.into();

    // Phase 1: Parse
    let program = match parser::parse_program(input_ts) {
        Ok(ast) => ast,
        Err(err) => return err.to_compile_error().into(),
    };

    // Phase 1.5: Semantic checks
    let mut diag = Diagnostic::new();
    check_variables(&program, &mut diag);
    check_break(&program.stmts, false, &mut diag);
    if let Err(err) = diag.emit() {
        return err.to_compile_error().into();
    }

    // Phase 2-4: IR → Codegen → Output
}

```

```
// ...  
}
```

11.6 调试模式

添加一个 `mlog_debug!` 宏，打印中间表示：

```
#[proc_macro]  
pub fn mlog_debug(input: TokenStream) -> TokenStream {  
    let program = match parser::parse_program(input.into()) {  
        Ok(ast) => ast,  
        Err(err) => return err.to_compile_error().into(),  
    };  
  
    let ir = ir::lower_program(&program);  
    let ir_dump = format!("{:?}", ir.instrs);  
    let mlog_text = codegen::generate_mlog(&ir.instrs);  
  
    let output = format!(  
        "/* IR dump:\n{}\n*/\n{}",  
        ir_dump, mlog_text  
    );  
  
    quote! { #output }.into()  
}
```

这样开发者在不确定编译器行为时可以用 `mlog_debug!` 看到中间表示的 `dump`。

11.7 小结

- 用 `Span` 精确定位 DSL 中的错误位置
- `Diagnostic` 系统收集多个错误一次报告
- 语义检查：未定义变量、不可变赋值、`break` 位置
- 检查在解析后、IR 生成前运行
- `mlog_debug!` 打印中间表示辅助调试
- 错误信息应该让 DSL 用户不需要看 `MLOG` 就能理解

12 实战：从 DSL 到 MLOG 的完整流程

 实验记录： 第十一站： 组装编译器

前面分别写了解析器、IR、代码生成、诊断。现在把它们拼在一起，并测试两个完整的 Mindustry 场景。

12.1 完整的宏入口

```
// mlog-macro/src/lib.rs
use proc_macro::TokenStream;

mod ast;
mod parser;
mod ir;
mod codegen;
mod diagnostics;

#[proc_macro]
pub fn mlog(input: TokenStream) -> TokenStream {
    let input_ts: proc_macro2::TokenStream = input.into();

    // 1. Parse DSL → AST
    let program = match parser::parse_program(input_ts) {
        Ok(ast) => ast,
        Err(err) => return err.to_compile_error().into(),
    };

    // 2. Semantic checks
    if let Err(err) = diagnostics::check(&program) {
        return err.to_compile_error().into();
    }

    // 3. AST → IR (Three-Address Code)
    let ir = ir::lower_program(&program);

    // 4. IR → MLOG text
    let mlog_text = codegen::generate_mlog(&ir.instrs);

    // 5. Return as &str literal
    quote::quote! { #mlog_text }.into()
}
```

12.2 场景一：仓库库存监控

在 Mindustry 中监控容器中铜的数量，小于阈值时启动传送带。

DSL:

```
let program = mlog! {
    let threshold = 100;

    loop {
        sensor(copper, container1, @copper);

        if copper < threshold {
            enable(conveyor1);
        } else {
```

```

        disable(conveyor1);
    }

    print("Copper: ");
    print(copper);
    print_flush(message1);
}
};

```

生成的 MLOG:

```

set __threshold_0 100
: __loop_1
sensor __copper_2 container1 @copper
op lt __t0 __copper_2 __threshold_0
jump __else_3 equal __t0 false
control enabled conveyor1 1 0 0 0
jump __end_if_4 always
: __else_3
control enabled conveyor1 0 0 0 0
: __end_if_4
print "Copper: "
print __copper_2
printf flush message1
jump __loop_1 always

```

验证: 复制到 Mindustry, 链接 container1、conveyor1 和 message1。运行后显示屏上会出现实时铜数量, 传送带在 < 100 时自动启动。

12.3 场景二: 单位巡逻

让一个 Poly 单位在两个坐标间来回巡逻。

DSL:

```

let program = mlog! {
    ubind(@poly);

    let target_x = 100;
    let target_y = 100;
    let mut at_first = true;

    loop {
        if at_first {
            target_x = 200;
            target_y = 300;
        } else {
            target_x = 100;
            target_y = 100;
        }

        ucontrol_move(target_x, target_y);

        sensor(x, @unit, @x);
        sensor(y, @unit, @y);
    }
}

```

```

    let dx = x - target_x;
    let dy = y - target_y;
    let dist = dx * dx + dy * dy;

    if dist < 16 {
        at_first = !at_first;
    }
}
};

```

生成的 MLOG:

```

ubind @poly
set __target_x_0 100
set __target_y_1 100
set __at_first_2 1
:__loop_3
jump __else_4 equal __at_first_2 false
set __target_x_0 200
set __target_y_1 300
jump __end_if_5 always
:__else_4
set __target_x_0 100
set __target_y_1 100
:__end_if_5
ucontrol move __target_x_0 __target_y_1 0 0 0
sensor __x_6 @unit @x
sensor __y_7 @unit @y
op sub __t0 __x_6 __target_x_0
set __dx_8 __t0
op sub __t1 __y_7 __target_y_1
set __dy_9 __t1
op mul __t2 __dx_8 __dx_8
op mul __t3 __dy_9 __dy_9
op add __t4 __t2 __t3
set __dist_10 __t4
op lt __t5 __dist_10 16
jump __skip_11 equal __t5 false
op eq __t6 __at_first_2 0
set __at_first_2 __t6
:__skip_11
jump __loop_3 always

```

🔍 核心直觉

注意 `!at_first` 的翻译: `at_first = !at_first` 在 MLOG 中变成:

```

op eq __t6 __at_first_2 0    // __t6 = (at_first == false)
set __at_first_2 __t6        // at_first = __t6

```

MLOG 没有逻辑非——用 `== false` (`eq x 0`) 模拟。

12.4 测试编译器的输出

使用单元测试验证生成的 MLOG:

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_simple_assignment() {
        let result = compile("let x = 5;");
        assert_eq!(result.trim(), "set __x_0 5");
    }

    #[test]
    fn test_addition() {
        let result = compile("let z = x + y;");
        assert_eq!(result.trim(),
            "op add __t0 __x_0 __y_1\nset __z_2 __t0");
    }

    #[test]
    fn test_while_loop() {
        let result = compile(
            "let mut i = 0; while i < 10 { i = i + 1; }"
        );
        assert!(result.contains(".__while_"));
        assert!(result.contains("jump"));
    }

    #[test]
    fn test_error_undefined_var() {
        let result = try_compile("let z = x;");
        assert!(result.is_err());
        assert!(result.unwrap_err().contains("not defined"));
    }
}
```

概念

单元测试验证生成 MLOG 的正确性，不需要 Mindustry。

把 compile 函数抽象出来（不含宏包装），在测试中直接调用解析 + IR + 代码生成。

12.5 编译器的文件清单

mlog-macro/src/	
├─ lib.rs	# 宏入口 (20 行)
├─ ast.rs	# AST 定义 (50 行)
├─ parser.rs	# 解析器 (350 行)
├─ ir.rs	# IR 定义 + 降级 (250 行)
├─ codegen.rs	# MLOG 代码生成 (150 行)
├─ diagnostics.rs	# 语义检查 + 错误报告 (100 行)
└─ vars.rs	# 作用域 + 变量管理 (80 行)

总共约 1000 行 Rust——一个完整的 MLOG 编译器。

12.6 小结

- 宏入口是装配线: 解析 → 诊断 → IR → 代码生成 → 输出
- 仓库监控: sensor + if/else + printflush 完整闭环
- 单位巡逻: ubind + ucontrol + 距离计算 + 状态切换
- MLOG 的 ! 用 eq x 0 模拟
- 单元测试验证 MLOG 输出正确性
- 完整编译器约 1000 行 Rust

13 总结与展望

✍ 实验记录： 终点站

13.1 回顾旅程

我们从一个问题出发——在 Mindustry 中手写 MLOG 太痛苦——到最终拥有一个嵌入在 Rust 中的 MLOG 编译器。

| 阶段 | 关键产出 | |:-|:-| | 理解 MLOG | 完整的指令集参考, 硬限制清单 | | DSL 设计 | Rust 语法 → MLOG 指令的映射表 | | 架构选型 | 函数式宏 + facade crate + 三阶段编译 | | 解析器 | 递归下降解析, 300 行 hand-rolled parser | | 中间表示 | 三地址码 IR, 表达式展平, 临时变量分配 | | 代码生成 | IR → MLOG 文本的直接映射 | | 控制流 | if/while/loop/break 的 jump 模式全解 | | 变量系统 | 作用域管理 + 唯一名称 + 不可变性检查 | | 错误诊断 | Span 精确定位 + 多错误收集 | | 完整编译器 | 仓库监控 + 单位巡逻两个实战案例 |

13.2 概念速查表

| 概念 | 在编译器中的位置 | |:-|:-| | TokenStream | 宏入口接收的输入 token 流 | | AST | 解析后的嵌套语法树 | | 三地址码 IR | MLOG op 指令的中间表示 | | 临时变量 | 表达式分解产生的 __tN | | 标签 | 控制流目标的 __while_N __break_N | | 唯一名称 | 用户变量的作用域安全映射 | | Span | 错误定位的源头 | | Diagnostic | 多错误收集器 | | quote! | 返回 MLOG 字符串字面量 |

13.3 与其他方案对比

| 方案 | 学习成本 | 集成 Rust | 类型安全 | 编译器控制 | |:-|:-|:-|:-|:-| | 手写 MLOG | 低 | N/A | 无 | 完全 | | Mindcode | 中 | 否 | 独立系统 | 间接 | | 本教程 (rust2mlog) | 低 (写 Rust) | 是 | 可扩展 | 完全 |

13.4 进阶扩展

13.4.1 优化器

在 IR 层面做优化:

- 死代码消除: 删除未使用的 set 和 op
- 公共子表达式消除: a + b 多次出现只算一次
- 常量折叠: 编译时计算的常量直接替换
- 临时变量压缩: 复用过期的临时变量

13.4.2 更多语言特性

- for i in 0..10 { ... } - 展开为 while 循环
- 数组访问 arr[index] - 映射到 read arr index
- 函数模拟 - 用标签 + jump 做内联“子程序”

13.4.3 编译错误的持续改进

- cargo expand 集成 - 查看展开后的 MLOG
- trybuild 集成 - 测试错误信息的质量
- LSP 支持 - VS Code 中跳转到错误位置

13.5 推荐阅读

- Mindcode 源码 - 另一个 MLOG 编译器的设计
- The MLOG ISA - MLOG 指令集的底层编码
- Crafting Interpreters (Nystrom) - 解析器和编译器的经典入门
- Engineering a Compiler (Cooper & Torczon) - 中间表示的深入讲解
- proc-macro-guide (本系列) - Rust 过程宏的全面教程

13.6 最后的思考

用 Rust 过程宏做编译器前端是一种被低估的模式。

传统编译器要么是独立工具 (gcc、clang)，要么是解释器 (Python eval)。过程宏提供了一种不同的路径——**编译时的程序变换**，可以直接嵌入在宿主语言的构建流程中。

本教程展示了这个模式的最简实例：一个约 1000 行的编译器，把类 Rust 的 DSL 编译成 Mindustry 汇编。

这不仅仅是关于 MLOG。任何需要“生成另一种语言的代码”的场景——SQL、着色器、配置文件、协议描述——都可以用同样的架构。

在 Rust 的世界里，只要你能 parse 它，你就能生成它。

